

Step 4 — A Fallback When a Server Is Unreachable, and a Full Working Demo — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints. Each has 3 depth levels: **Basic** (the plain idea), **Intermediate** (the real `asyncio` + `try/except` shapes), **Advanced** (the part that's easy to get half-right: a server failing *mid-task*, after it already connected fine). Read Basic first — the core idea (don't let one bad server take down the rest) is simple; the depth is in doing it at both the right moments (connect time, *and* call time).

- [Hint 1 — Two different failure moments, and why they need two different fixes](#)
- [Hint 2 — The pool's resilience layer, and the final demo](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Two different failure moments, and why they need two different fixes

Basic Version

There are two completely different moments a server can fail, and they need two different fixes. The first: a server never comes up at all — wrong command, crashed on startup, hangs forever without ever finishing the handshake. That has to be caught **while connecting**, before the agent ever starts. The second: a server connects fine and works for a while, then dies or drops the connection partway through a task — a tool call that used to work suddenly raises. That has to be caught **at the moment of the call itself**, every single time, not just once at startup.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

For the connect-time failure: wrap each server's connection attempt (from Step 3's `connect_all`, one server at a time, inside the `for` loop) in `asyncio.wait_for(..., timeout=...)` and a `try / except`. On failure, log it clearly and `continue` to the next server — don't let one bad entry in `servers_config` stop the loop from even trying the rest:

```
try:
    read, write = await asyncio.wait_for(
        self._stack.enter_async_context(stdio_client(params)), timeout=10,
    )
    session = await asyncio.wait_for(
        self._stack.enter_async_context(ClientSession(read, write)), timeout=10,
    )
    await asyncio.wait_for(session.initialize(), timeout=10)
except Exception as e:
    print(f"[{server['name']}] unreachable: {type(e).__name__}: {e} -- continuing")
    continue
```

For the call-time failure: wrap the actual dispatch inside `call_tool` (Step 3's `MCPServerPool.call_tool`) in its own `try / except`, and — critically — **return** a clearly-labeled error string instead of letting the exception propagate. The agent's loop already knows how to feed a tool's returned string back to the model as an observation (Step 2); a server dying mid-task should look, to that loop, exactly like a tool that returned an error, not like a crash:

```
try:
    return await call_mcp_tool(session, original_name, arguments)
except Exception as e:
    return f"Error: the '{server_name}' server is not responding ({type(e).__name__})"
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what happens on the *second* call to a server that already failed once mid-task. The fix in Intermediate catches the failure and reports it cleanly — but if the same broken connection is tried again on the very next tool call, you get the same failure again, one more wasted round-trip (and, worse, it can push the model toward repeating a near-identical tool call, exactly the "infinite loop" failure Doc07 names). The cleaner design: the *first* failure from a given server marks that server as dead for the rest of the run, and every subsequent lookup skips it immediately — no second attempt, no second timeout wait.

This means the tool list itself needs to be able to shrink mid-run. Up to now, `openai_tools` has been built once, before the agent loop starts, and handed to every model call unchanged. Once a server can go dead partway through, the agent loop needs to ask the pool for the *current* available tools on every iteration, not reuse the list from the very first call — otherwise the model keeps being offered tools that the pool has already learned are broken.

Sketch how you'd track "servers that are currently dead" and how `agent.py`'s loop would need to change to ask for a fresh tool list each iteration, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the two failure moments. Intermediate gives the real fix for each — a timeout-wrapped connect, and a caught, labeled call — treating every call as an independent, retryable event. Advanced adds the *state* that makes this efficient rather than just correct: remembering a server is dead so it's never retried, and reflecting that in the tools actually offered to the model on later steps of the same run, not just at the very start.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The pool's resilience layer, and the final demo

Basic Version

```
pool keeps a set of "dead" server names, starts empty

connect_all: for each server, try connecting with a timeout
    if it fails: log it, add nothing for it, move on to the next server
    if it works: store its session, store its tools, remember its full
                  openai-schema list

get_current_tools(): return the stored tools list, minus any whose
                    server is in the "dead" set

call_tool(name, args):
    look up which server this name belongs to
    if that server is already marked dead: return a canned "unavailable"
                                           message immediately, no attempt
    otherwise: try the real call
        if it fails: mark that server dead, return a labeled error
        if it works: return the result normally

main.py: connect, then loop the agent, each iteration asking the pool for
        get_current_tools() fresh instead of reusing one list from the start
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# server_pool.py -- building on Step 3's MCPServerPool
import asyncio

class MCPServerPool:
    def __init__(self) -> None:
        self._sessions = {}
        self._routing = {}
        self._all_tools = []          # every tool discovered from every server to
        self._dead_servers = set()
        self._stack = AsyncExitStack()

    async def connect_all(self, servers_config: list[dict], connect_timeout: float):
        # your turn: same duplicate-name check as Step 3, then for each
        # server, wrap the connect attempt in try/except + asyncio.wait_for,
        # `continue` past a failed one instead of raising, and append
        # everything that DOES connect's tools into self._all_tools
        ...
        return self.get_current_tools()

    def get_current_tools(self) -> list[dict]:
        return [
            t for t in self._all_tools
            if self._routing[t["function"]["name"]][0] not in self._dead_servers
        ]

    async def call_tool(self, namespaced_name: str, arguments: dict) -> str:
        server_name, original_name = self._routing[namespaced_name]
        if server_name in self._dead_servers:
            return f"Error: the '{server_name}' server is unavailable for the request"
        session = self._sessions[server_name]
        try:
            return await call_mcp_tool(session, original_name, arguments)
        except Exception as e:
            self._dead_servers.add(server_name)
            return f"Error: the '{server_name}' server is not responding ({type(e).__name__})"

    async def close(self) -> None:
        await self._stack.aclose()
```

And `agent.py`'s loop needs exactly one change from Step 3 — ask for a fresh tool list each time through, instead of reusing whatever was passed in at the start:

```
# inside run_agent's for-loop, before calling the model:  
current_tools = pool.get_current_tools()  
response = client.chat.completions.create(model="gpt-4o-mini", messages=messages,
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Fill in `connect_all`, then write the final `main.py` demo: a `servers_config.py` with one entry deliberately pointing at a script that doesn't exist, run both of **A Real Example**'s tasks 3 and 4, and print a clear summary at the end — which servers connected, which didn't, and the agent's final answer either way. Then write `test_agent.py`: rather than actually killing a subprocess (fragile to set up reliably in a short test), simulate a mid-task drop by replacing one connected session's `call_tool` method with a function that always raises, and confirm `pool.call_tool(...)` returns a labeled error string (not an exception), that the server is now in `pool._dead_servers`, and that `pool.get_current_tools()` no longer includes its tools.

Compare your finished version against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic is the plan, no code. Intermediate gives the real `MCPServerPool` resilience methods and the one-line change `agent.py` needs, with the connect loop itself left for you to fill in from Hint 1's Intermediate section. Advanced adds the actual `main.py` demo and a real (if simulated) test proving the fallback path, not just the happy path.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)